

To save the changes, call the commit method and finally close the connection, as shown below:

```
connection.commit()

connection.close()
```

Now the code looks like what's shown below:

```
import sqlite3
connection = sqlite3.connect("College.db")
cursor = connection.cursor()

cmd = """CREATE TABLE employee (ssn INTEGER PRIMARY KEY,
fname VARCHAR(20), lname VARCHAR(30), salary number,
birth_date DATE);"""
cursor.execute(cmd)
cmd = """INSERT INTO employee VALUES ("12398", "John",
"Danny", "80000", "1961-10-25");"""
cmd = """INSERT INTO employee VALUES ("98765", "Ryan",
"Harry", "65000", "1965-03-04");"""

connection.commit()
connection.close()
```

The Python code to query our employee table is as follows:

```
import sqlite3
connection = sqlite3.connect("College.db")
cursor = connection.cursor()
cursor.execute("SELECT * FROM employee")
print("The employee details : ")

#We use .fetchmany() to load optimal no of rows and
overcome memory issues in case of large datasets
#In case the number of rows in the table is small, you
can use the fetchall() method to fetch all rows from the
database table

result = cursor.fetchall()
for res in result:
    print(res)
cursor.execute("SELECT * FROM employee")
print("\nfetch one record:")

#fetchone() returns one record as a tuple or it fetches
the records one by one, If there are no more records then
it returns None

res_one = cursor.fetchone()
print(res_one)
```

The output returned is:

```
The employee details:
('12398', 'John', 'Danny', '80000', '1961-10-25')
('98765', 'Ryan', 'Harry', '65000', '1965-03-04')

fetch one record:
('12398', 'John', 'Danny', '80000', '1961-10-25')
```

A brief summary of the code

- First, we connected to the database by creating a new connection object.
- Second, from the connection object, we instantiated a new cursor object.
- Third, we executed a query that selects all rows from the *books* table.
- Fourth, we called the *fetchone()* method to fetch the next row in the result set.
- Fifth, we closed both cursor and connection objects by invoking the *close()* method of the corresponding object.

Reading relational tables

We pick Sqlite3 as our relational database as it is very lightweight and easy to use. To begin with, create a database engine and then connect to it using the *to_sql* function of the SQLAlchemy library.

The relational table is created by using the *to_sql* function from a DataFrame already created, by reading a CSV file. Then we use the *read_sql_query* function to read the results of an SQL query directly into a Pandas DataFrame, as shown below:

```
from sqlalchemy import create_engine
import pandas as pd

data = pd.read_csv('C:/Users/Neethu/input.csv')

# To connect we use create_engine(): Create the db engine
# we will use an in-memory-only SQLite database
engine = create_engine('sqlite:///memory:')
#The return value of create_engine() is an instance of
Engine, and it represents the core interface to the
database

# Store the dataframe as a table
data.to_sql('data_table', engine)

# Query 1 on the relational table
res1 = pd.read_sql_query('SELECT * FROM data_table',
engine)
print('Result 1', res1)
```